FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Eric V. Tividad Jr. Jose Miguel B. Viado
Section:	AN 42
Professor:	Ms. Crisola Tan
Date:	10/4/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.
Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking
✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation
✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System
A CBR system identifies the learning struggles of students. Student submits a math solution with repeated errors. System retrieves past students with similar error patterns (e.g., misusing formulas). It adapts feedback and personalized lessons based on successful interventions. **Learning:** System improves by retaining new student profiles and outcomes.

CASE-BASED REASONING (CBR) DISASTER RESPONSE DECISION SUPPORT SYSTEM:

Disaster Response Decision Support System that uses Case-Based Reasoning. It compares a new disaster with previous cases, retrieves the most similar case, adapts its solution, and saves the new case for future use.

CASE BASE:

The program contains six previous disaster cases: Typhoon Ursula, Typhoon Odette, Flooding Ulysses, Flooding Ondoy, Bohol Earthquake, and Moro Gulf Earthquake and Tsunami. Each case contains disaster type, area, flood risk, population, and a previous response consisting of evacuees, shelters, boats and relief packs.

SIMILARITY METHOD:

The program compares the new disaster with every case using four attributes: disaster type (35%), area (25%), flood risk (20%), and population (20%). The case with the highest similarity score is selected as the most similar disaster.

ADAPTATION LOGIC:

The retrieved solution is adjusted according to the population of the new disaster. The number of evacuees, shelters, boats, and relief packs are scaled using the population ratio. If the new disaster has a higher flood risk, evacuees and boats are increased by 20%. The user can then accept or modify the generated solution.

Output:

```
TA3: Disaster Response Decision Support System

ENTER THE NEW DISASTER TO PREPARE FOR:
Disaster type (Typhoon/Flood/Earthquake): Flood
Area type (Urban/Coastal/Rural): Urban
Flood risk (1 = Low, 2 = Medium, 3 = High): 2
Population (numbers only): 2000

NEW PROBLEM TO PREPARE FOR:
Disaster type: Flood
Area type: Urban
Flood risk: 2 (1 = Low, 2 = Medium, 3 = High)
Population: 2000
```

```
DISASTER SIMILARITY SEARCH:
Typhoon Ursula (2019) -> 0.36
Typhoon Odette (2021) -> 0.1
Flooding Ulysses (2020) -> 0.7
Flooding Ondoy (2009) -> 0.7
Bohol Earthquake (2013) -> 0.11
Moro Gulf Earthquake and Tsunami (1976) -> 0.1

Most Similar Disaster: Flooding Ulysses (2020) (score: 0.7)
Old solution: {'evacuees': 27000, 'shelters': 9, 'boats': 10, 'relief_packs': 27000}
```

```
SOLUTION ADAPTATION:
Population ratio: 0.02
Adapted solution: {'evacuees': 600, 'shelters': 0, 'boats': 0, 'relief_packs': 600}
```

```
SOLUTION ADAPTATION: FINALIZATION:
Would like to change the new solution? (y/n): y
Enter new evacuees (Enter to keep 600): 1000
evacuees changed to 1000
Enter new shelters (Enter to keep 0): 20
shelters changed to 20
Enter new boats (Enter to keep 0): 20
boats changed to 20
Enter new relief_packs (Enter to keep 600): 20
relief_packs changed to 20

FINAL SOLUTION: {'evacuees': 1000, 'shelters': 20, 'boats': 20, 'relief_packs': 20}

FINAL SOLUTION FOR THE NEW DISASTER:
New disaster case saved!

UPDATED CASE BASE:
1. Typhoon Ursula (2019)
2. Typhoon Odette (2021)
3. Flooding Ulysses (2020)
4. Flooding Ondoy (2009)
5. Bohol Earthquake (2013)
6. Moro Gulf Earthquake and Tsunami (1976)
7. New Flood Case #7 (Urban, 2000)
Total cases: 7
```

Your Code - Laboratory Tasks ⇩

```
# Steps of the program:
# 1. Similarity search              - find the most similar past disaster for the new disaster
# 2. Solution adaptation             - adapt the old solution to the new problem
# 3. Solution adaptation: finalization  - user can either accept or change the solution
# 4. Final solution                 - save the new case in the case base

# past disasters we will use to compare
# each has a problem and solution dictionary, for attribute comparison stuff
# types: Typhoon, Flood, and Earthquakes
# area: Urban, Coastal, and Rural
# flood_risk: 1 - low, 2 - medium, 3 - high risk level
case_base = [
   {"name": "Typhoon Ursula (2019)",
    "problem": {"type": "Typhoon",
                "area": "Urban",
                "flood_risk": 3,
                "population": 60000},
```

```
 "solution": {"evacuees": 18000,
              "shelters": 6,
              "boats": 6,
              "relief_packs": 18000}},

{"name": "Typhoon Odette (2021)",
 "problem": {"type": "Typhoon",
             "area": "Coastal",
             "flood_risk": 3,
             "population": 120000},

 "solution": {"evacuees": 48000,
              "shelters": 16,
              "boats": 12,
              "relief_packs": 48000}},

{"name": "Flooding Ulysses (2020)",
 "problem": {"type": "Flood",
             "area": "Urban",
             "flood_risk": 3,
             "population": 90000},

 "solution": {"evacuees": 27000,
              "shelters": 9,
              "boats": 10,
              "relief_packs": 27000}},

{"name": "Flooding Ondoy (2009)",
 "problem": {"type": "Flood",
             "area": "Urban",
             "flood_risk": 3,
             "population": 150000},

 "solution": {"evacuees": 60000,
              "shelters": 20,
              "boats": 15,
              "relief_packs": 60000}},

{"name": "Bohol Earthquake (2013)",
 "problem": {"type": "Earthquake",
             "area": "Rural",
             "flood_risk": 1,
```

```
            "population": 40000},

     "solution": {"evacuees": 12000,
                  "shelters": 5,
                  "boats": 0,
                  "relief_packs": 12000}},

    {"name": "Moro Gulf Earthquake and Tsunami (1976)",
     "problem": {"type": "Earthquake",
                 "area": "Coastal",
                 "flood_risk": 3,
                 "population": 90000},

     "solution": {"evacuees": 45000,
                  "shelters": 15,
                  "boats": 8,
                  "relief_packs": 45000}}
]

# determines if the new typhoon is similar to the past typhoons
# similarity weight: type = 0.35, area = 0.25, flood risk = 0.20, population = 0.20
def get_similarity(new_problem, old_problem):
    score = 0

    # disaster type similarity
    if new_problem["type"] == old_problem["type"]:
        score = score + 0.35

    # area type similarity
    if new_problem["area"] == old_problem["area"]:
        score = score + 0.25

    # flood risk similarity (the closer, the higher)
    difference = abs(new_problem["flood_risk"] - old_problem["flood_risk"])
    score = score + 0.20 * (1 - difference / 2)

    # population similarity (the closer, the higher)
    smaller = min(new_problem["population"], old_problem["population"])
    bigger = max(new_problem["population"], old_problem["population"])
    score = score + 0.20 * (smaller / bigger)

    return score
```

```
print("TA3: Disaster Response Decision Support System")

while True:
    # the new disaster to analyze, the user types it in
    print("")
    print("ENTER THE NEW DISASTER TO PREPARE FOR:")

    # disaster type
    while True:
        new_type = input("Disaster type (Typhoon/Flood/Earthquake): ").strip().capitalize()
        if new_type in ["Typhoon", "Flood", "Earthquake"]:
            break
        print("Please enter valid disaster type!")

    # area type
    while True:
        new_area = input("Area type (Urban/Coastal/Rural): ").strip().capitalize()
        if new_area in ["Urban", "Coastal", "Rural"]:
            break
        print("Please enter valid area type!")

    # flood risk (1, 2, or 3 only)
    while True:
        risk_text = input("Flood risk (1 = Low, 2 = Medium, 3 = High): ").strip()
        if risk_text in ["1", "2", "3"]:
            new_risk = int(risk_text)
            break
        print("Please enter valid levels!")

    # population (whole number greater than 0, commas are removed)
    while True:
        population_text = input("Population (numbers only): ").strip().replace(",", "")
        if population_text.isdigit() and int(population_text) > 0:
            new_population = int(population_text)
            break
        print("Please enter a number greater than 0")

    new_problem = {"type": new_type,
                   "area": new_area,
                   "flood_risk": new_risk,
                   "population": new_population}
    print("")
    print("NEW PROBLEM TO PREPARE FOR:")
```

```
print("Disaster type: " + new_problem["type"])
print("Area type: " + new_problem["area"])
print("Flood risk: " + str(new_problem["flood_risk"]) + " (1 = Low, 2 = Medium, 3 = High)")
print("Population: " + str(new_problem["population"]))

# step 1: determine which typhoon is similar to the new one
print("")
print("DISASTER SIMILARITY SEARCH:")
best_case = None
best_score = -1

for case in case_base:
    score = get_similarity(new_problem, case["problem"])
    print(case["name"] + " -> " + str(round(score, 2)))
    # keep the case with the highest score
    if score > best_score:
        best_score = score
        best_case = case

print("")
print("Most Similar Disaster: " + best_case["name"] + " (score: " + str(round(best_score, 2)) + ")")
old_solution = best_case["solution"]
print("Old solution: " + str(old_solution))

# step 2: adapting the old solution to the new disaster
print("")
print("SOLUTION ADAPTATION:")

# gets the population ratio of new and similar disaster
ratio = new_problem["population"] / best_case["problem"]["population"]
print("Population ratio: " + str(round(ratio, 2)))

new_solution = {}
new_solution["evacuees"] = round(old_solution["evacuees"] * ratio)
new_solution["shelters"] = round(old_solution["shelters"] * ratio)
new_solution["boats"] = round(old_solution["boats"] * ratio)
new_solution["relief_packs"] = round(old_solution["relief_packs"] * ratio)

# if the flood risk is higher than the old case, add 20% evacuees and boats
if new_problem["flood_risk"] > best_case["problem"]["flood_risk"]:
    new_solution["evacuees"] = round(new_solution["evacuees"] * 1.2)
    new_solution["boats"] = round(new_solution["boats"] * 1.2)
    print("Flood risk is higher, evacuees and boats increased by 20%")
```

```
print("Adapted solution: " + str(new_solution))

# step 3: use or revise the provided solution
print("")
print("SOLUTION ADAPTATION: FINALIZATION:")
answer = input("Would like to change the new solution? (y/n): ").strip().lower()

if answer == "y":
    for item in ["evacuees", "shelters", "boats", "relief_packs"]:
        number = input("Enter new " + item + " (Enter to keep " + str(new_solution[item]) + "): ").strip()
        if number.isdigit():
            new_solution[item] = int(number)
            print(item + " changed to " + number)
        elif number != "":
            print("Invalid number, " + item + " not changed")
else:
    print("Solution accepted")

print("")
print("FINAL SOLUTION: " + str(new_solution))

# step 4: final solution
print("")
print("FINAL SOLUTION FOR THE NEW DISASTER:")
new_case = {"name": "New " + new_problem["type"] + " Case #" + str(len(case_base) + 1) +
                    " (" + new_problem["area"] + ", " + str(new_problem["population"]) + ")",
            "problem": new_problem,
            "solution": new_solution}
case_base.append(new_case)
print("New disaster case saved!")

print("")
print("UPDATED CASE BASE:")
number = 1
for case in case_base:
    print(str(number) + ". " + case["name"])
    number = number + 1
print("Total cases: " + str(len(case_base)))

# continue or no?
print("")
again = input("Analyze another disaster? (y/n): ").strip().lower()
```

```
if again == "n":
    print("Goodbye!")
    break
```

Pair Reflection:

1. What did each partner contribute?
    - Both partners contributed equally. One wrote the program's source code, while the other one wrote the documentation section and compiled the program's output.

2. What challenges have you encountered?
    - In terms of conceptualizing the program, I think it'll be designing the logic and deciding which attributes suit better. For example, the adaptation of the old solution doesn't mean just use it exactly as it is, as the new disaster has some specific things that either need more or less. As for the attribute delegation, flood risk is the one we're pretty suspicious about, as earthquakes don't always cause floods. But after some research, it can indirectly cause a flood. If it did not cause one though, earthquake disasters will have a distinct attribute compared to the other two types of disaster we have.

3. What you learned about CBR?
    - I think CBR systems just reflect the "learning from the past" mindset we have, and it's very useful in various applications. We recall a similar past situation, adjust what worked to fit the new one, and remember the result for next time. Though it may seem just a simple look back at things, it can help us gain a ton of helpful information that can be applied in many ways. One thing that we're not always ready for is the uncertainty that may or may not happen every day. It could either be bad weather or a bad interaction we would have; either way, we're unprepared for it. However, with enough experience, we can greatly adapt to such situations and move forward like we have handled it beforehand.